

CS683 2026 Assignment 1 Task 2 Report

24B1020, 24B1046, 24B1078, 24B1082

September 2026

1 Methodology

Three kernels from `../src/` – `matmul_prefetch` (software prefetching via `mm_prefetch`, tunable at compile time through `-DPF_DIST/-DPF_HINT_ID`), `matmul_simd` (128-bit and 256-bit AVX2 explicit intrinsics, selected by the kernel’s own `FLAG_128` branch), and `matmul_optimized` (a 2×4 register-blocked, $MC \times NC$ cache-tiled kernel that combines 256-bit AVX2 SIMD with software prefetching) – were benchmarked against the `matmul_naive` reference on identical, randomly generated inputs, with every result checked against `matmul_naive`’s output before being recorded. Each configuration (technique $\times$ matrix size $\times$ parameter) was run with 1 untimed warmup followed by 10 timed repetitions, and the reported speedup is the mean across those 10 runs. Square matrices of size $N = 256, 512$, and 1024 were tested. `matmul_prefetch` and `matmul_optimized` were additionally swept over their own tunable prefetch-distance parameter (`PF_DIST` for the former, `PF_ROWS` for the latter, each in the kernel’s own unit) and over four prefetch cache-fill hints (`T0/T1/T2/NTA`), both sweeps done at a fixed representative size of $N = 512$. All binaries were compiled with `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`, so that SIMD gains are attributable only to the explicit intrinsics in each kernel and not to compiler auto-vectorization. Cache-miss and instruction counts were collected with Valgrind (`cachegrind` and `callgrind`) rather than `perf`, since `perf_event_open` is blocked under the WSL2 hypervisor used for development; these profiling runs were kept to smaller sizes ($N = 64, 128, 256$) to keep their overhead manageable, and `cachegrind`’s D1 miss count was used as the cache-miss proxy since it is the counter the tool honours exactly for a manually specified cache geometry.

We were unable to see the effect of the hardware prefetcher for this assignment, as all 4 members of the team did not have the provision to disable hardware prefetching on the laptop.

2 Results

Tables 1–4 give the mean speedup over `matmul_naive` for each technique and sweep, and Figures 1–9 plot the same data together with the cache-miss and instruction-count measurements.

Table 1: Mean speedup vs. `matmul_naive` by matrix size, all techniques.

N	prefetch	simd (128-bit)	simd (256-bit)	optimized (256-bit)
256	0.96	5.05	10.80	32.30
512	0.92	4.55	9.65	32.82
1024	1.01	3.81	7.66	29.60

Table 2: Task 2A: `matmul_prefetch` speedup vs. `PF_DIST` ($N = 512$, `hint=T0`).

<code>PF_DIST</code>	speedup
0	1.02
8	0.98
16	0.94
32	0.95
64	0.97
128	1.08
256	0.89

Table 3: Task 2C: `matmul_optimized` speedup vs. `PF_ROWS` ($N = 512$, `hint=T0`).

<code>PF_ROWS</code>	speedup
0	31.32
4	32.56
8	30.27
16	33.57
32	25.11
64	35.94
128	29.14

Table 4: Speedup vs. prefetch cache-fill hint ($N = 512$), `matmul_prefetch` (PF_DIST=64) vs. `matmul_optimized` (PF_ROWS=8).

hint	prefetch speedup	optimized speedup
T0	1.05	33.41
T1	0.96	31.40
T2	0.99	32.18
NTA	0.97	31.23

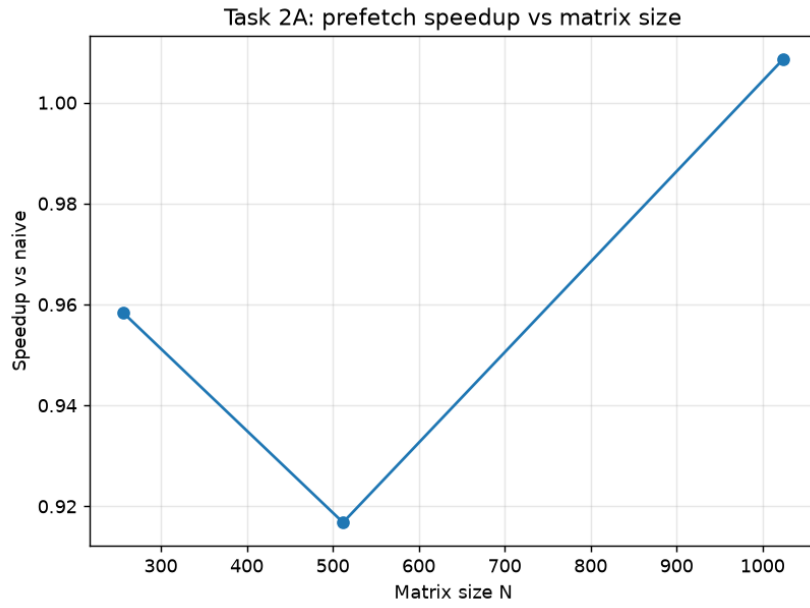


Figure 1: Task 2A: `matmul_prefetch` speedup vs. matrix size (PF_DIST=64, hint=T0).

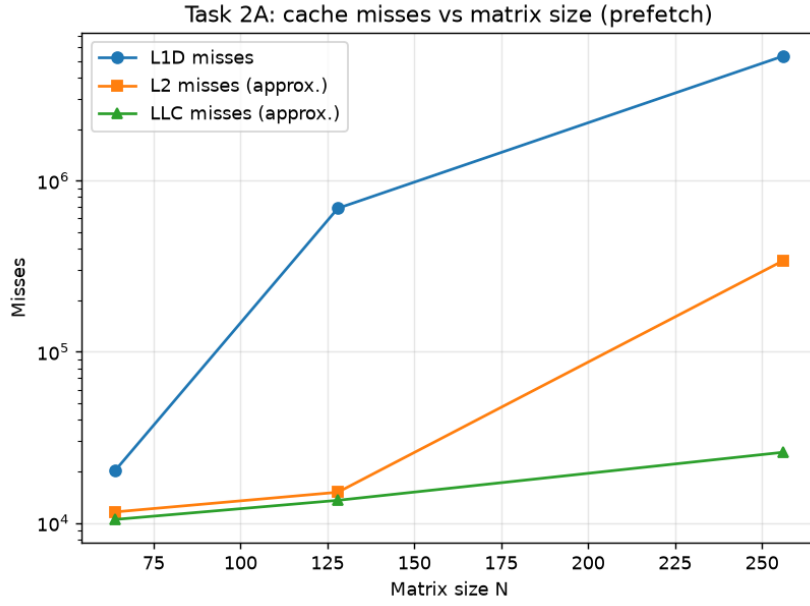


Figure 2: Task 2A: L1D/L2/LLC cache misses (cachegrind, 3-pass D1 approximation) vs. matrix size, `matmul_prefetch`.

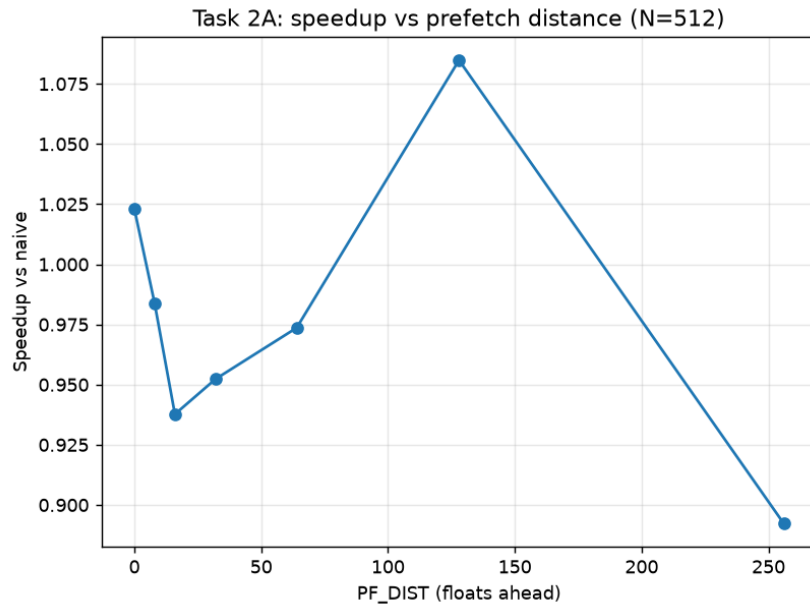


Figure 3: Task 2A: `matmul_prefetch` speedup vs. PF_DIST ($N = 512$, `hint=T0`).

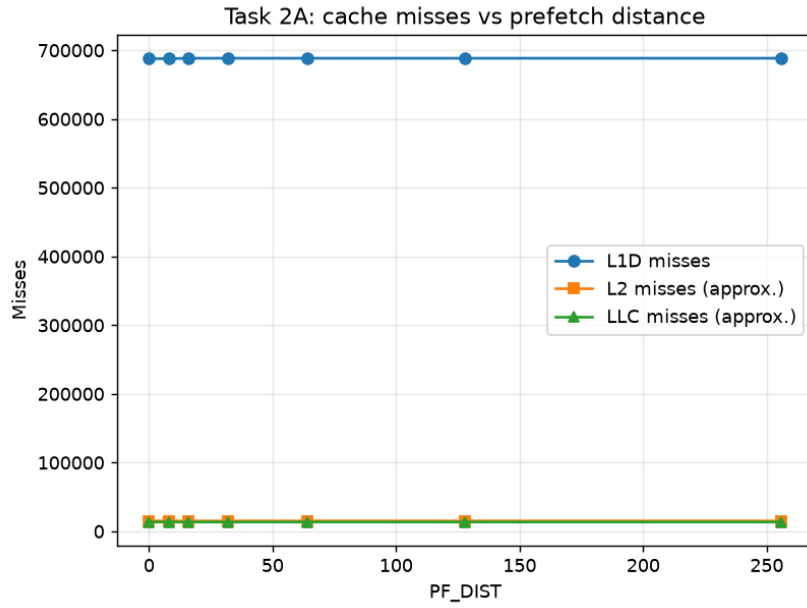


Figure 4: Task 2A: cache misses vs. PF_DIST ($N = 128$, hint=T0).

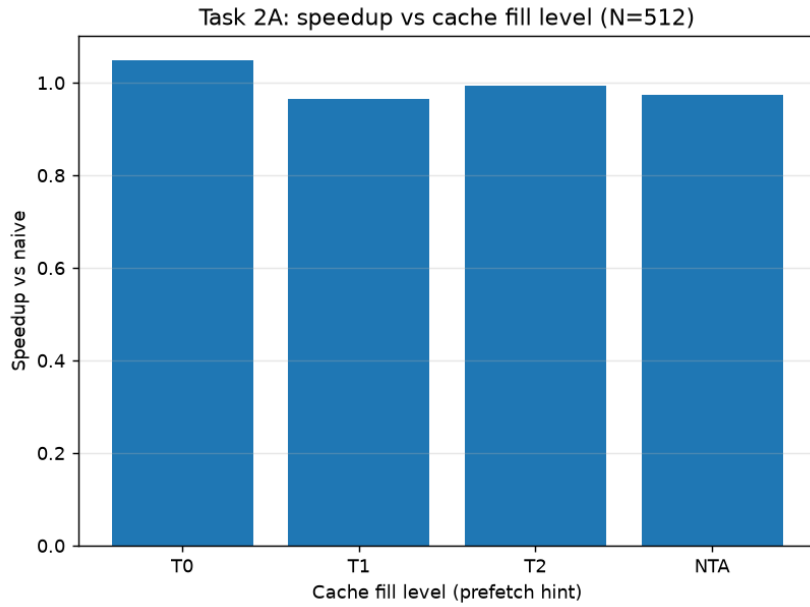


Figure 5: Task 2A: `matmul_prefetch` speedup vs. prefetch cache-fill hint ($N = 512$, PF_DIST=64).

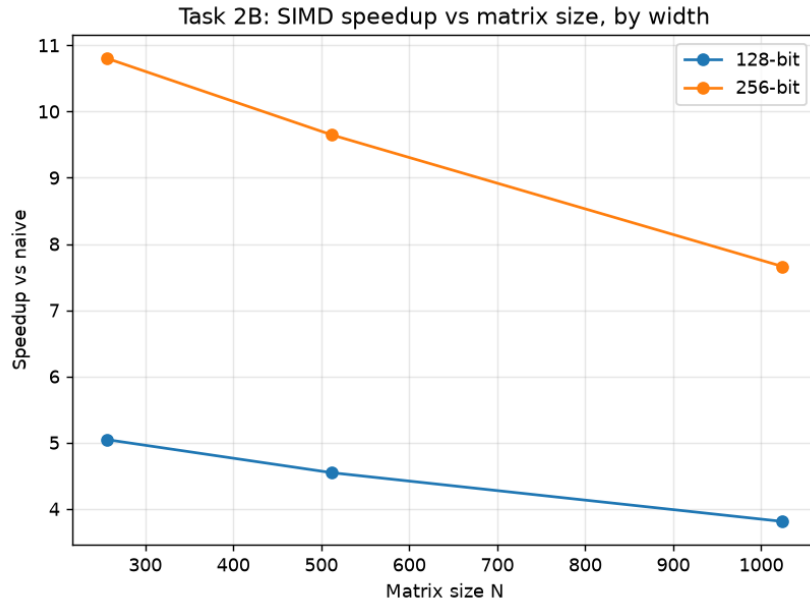


Figure 6: Task 2B: `matmul_simd` speedup vs. matrix size, 128-bit vs. 256-bit AVX2.

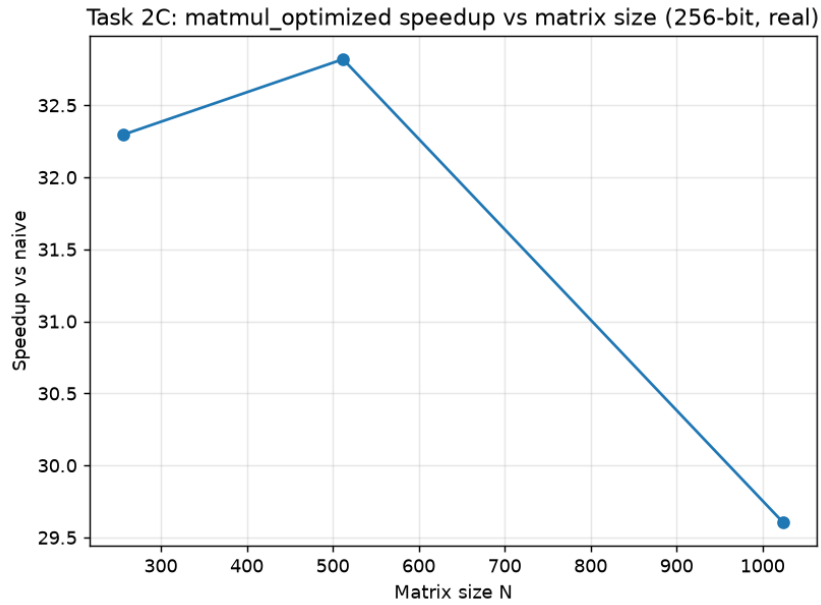


Figure 7: Task 2C: `matmul_optimized` speedup vs. matrix size (256-bit, PF_ROWS=8, hint=T0).

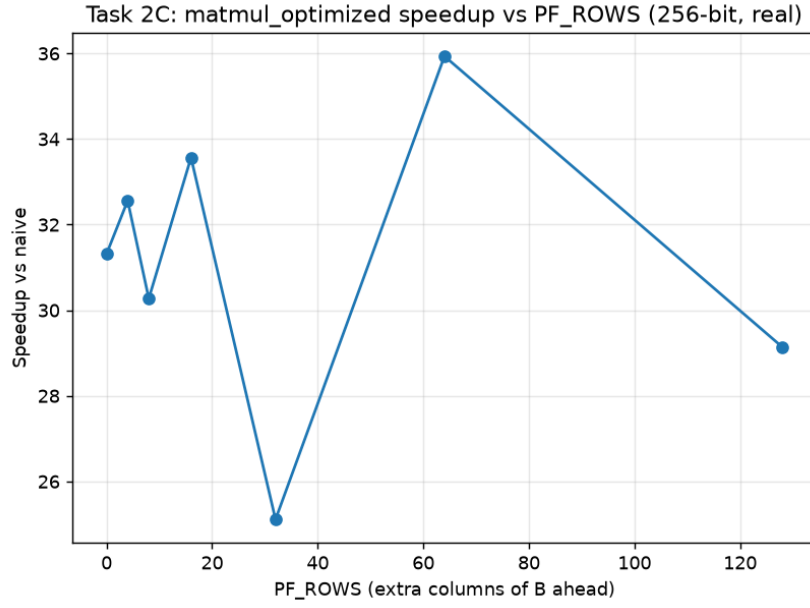


Figure 8: Task 2C: matmul_optimized speedup vs. PF_ROWS ($N = 512$, hint=T0).

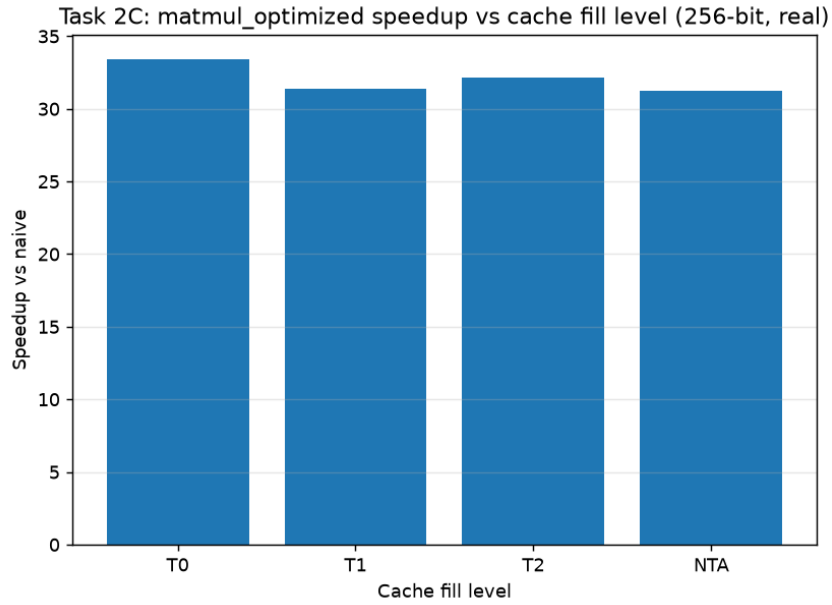


Figure 9: Task 2C: matmul_optimized speedup vs. prefetch cache-fill hint ($N = 512$, PF_ROWS=8).

3 Discussion

Software prefetching (`matmul_prefetch`)

Software prefetching without disabling the hardware prefetcher gives essentially no benefit in this kernel: the measured speedup over `matmul_naive` sits at $0.92\times$ – $1.01\times$ across $N = 256, 512, 1024$ (Table 1), i.e. statistically indistinguishable from $1\times$ given run-to-run noise, and none of the seven `PF_DIST` values tested (Table 2) or four cache-fill hints (Table 4) push it consistently above about $1.08\times$. The cachegrind data explains why: at $N = 256$, `matmul_prefetch`’s D1 miss count (5,323,303) is essentially identical to `matmul_naive`’s (5,322,024, well under 0.1% apart, within simulation noise), meaning the explicit `mm_prefetch` calls are not preventing any additional cache misses at this problem size (Figures 2, 4). This is consistent with the kernel still being a scalar, untiled triple loop: the hardware prefetcher, already active on any modern x86 core independent of software hints, was likely already covering most of the exploitable locality, leaving little room for a manual hint to add anything.

SIMD vectorization (`matmul_simd`)

SIMD gives the single largest per-technique speedup: up to $10.80\times$ (256-bit) and $5.05\times$ (128-bit) at $N = 256$, falling to $7.66\times/3.81\times$ by $N = 1024$ as the working set outgrows cache and the vector unit becomes increasingly starved of data (Table 1, Figure 6). The 256-bit path is consistently about $2\times$ faster than 128-bit at every size tested ($10.80/5.05 = 2.14\times$ at $N = 256$, $9.65/4.55 = 2.12\times$ at $N = 512$, $7.66/3.81 = 2.01\times$ at $N = 1024$), which tracks almost exactly the doubling of floats-per-register (8 vs. 4), confirming the win scales with vector width as expected. At $N = 256$ the 256-bit kernel executes 336,480,271 instructions against `matmul_naive`’s 515,526,869. Vectorization is not shrinking the instruction stream by much (loop bookkeeping, address computation, and any scalar remainder handling do not vectorize away), so the bulk of the observed speedup must come from a lower cycles-per-instruction rather than fewer instructions – consistent with the D1-miss count at $N = 256$ being effectively unchanged from naive (5,322,027 for 256-bit SIMD vs. 5,322,024 for naive): wider loads let each retired instruction do more useful work on data that was going to be pulled into cache anyway, rather than reducing traffic to memory.

Combined optimization (`matmul_optimized`)

The fully combined kernel (2×4 register blocking, $MC \times NC$ cache tiling, 256-bit AVX2, and software prefetching) dominates every individual technique at every size tested: $29.60\times$ – $32.82\times$ (Table 1, Figure 7), roughly 3–4 $\times$ higher than SIMD alone despite executing only marginally fewer instructions than the SIMD-only kernel (324,514,381 vs. 336,480,271 at $N = 256$, a mere $1.04\times$ difference). The extra speedup is explained by cache behaviour rather than instruction count: `matmul_optimized`’s D1 misses at $N = 256$ (4,282,460) are about 20% lower than

the untiled SIMD/naive/prefetch kernels' $\approx 5.32\text{M}$. This matches the expectation that tiling and register blocking, not vectorization or prefetching in isolation, are what actually reduce memory traffic. Both of the combined kernel's tunable parameters are comparatively flat once away from pathological settings: `PF_ROWS` swings between $25.11\times$ (worst, `PF_ROWS=32`) and $35.94\times$ (best, `PF_ROWS=64`) at $N = 512$ with no clear monotonic trend (Table 3, Figure 8) – at these already-small absolute run times (4–7 ms) point-to-point noise is a larger fraction of the signal than it was for `matmul_prefetch`'s much slower runs – and the cache-fill hint again favours T0 only marginally ($33.41\times$ vs. $31.2\text{--}32.2\times$ for the other three hints, Table 4, Figure 9), mirroring the same weak hint sensitivity seen for `matmul_prefetch` alone.